

Software-Qualität

Aufgabenblatt 3

Julian Bertol
12. April 2026

Risikobasiertes Testen nutzt Informationen über Projekt- und Produktrisiken und legt den Testschwerpunkt auf Bereiche hohen Risikos. Risiko wird als Produkt aus Schadensausmaß und Schadenswahrscheinlichkeit definiert. Testfälle werden entsprechend priorisiert: Bereiche mit hohem Risiko werden intensiver und früher getestet, unkritische Bereiche erhalten weniger Aufwand. Dies verhindert eine ineffiziente Gleichverteilung der Testressourcen.

Wiederverwendungsorientiertes Testen übernimmt vorhandene Tests und Testumgebungen aus früheren Projekten als Ausgangsbasis. Ziel ist es, den Testbetrieb schnell aufzusetzen. Es setzt stark auf Testautomatisierung und Regressionstestsuiten, um bewährte Testfälle effizient wiederholen zu können.

Checklistenbasierter Ansatz nutzt Fehlerlisten aus früheren Testzyklen, Listen potenzieller Fehler oder priorisierter Qualitätskriterien. Er ist wenig formal, aber pragmatisch: Bekannte Fehlerquellen werden systematisch abgearbeitet. Wo früher Fehler aufgetreten sind, sind häufig weitere zu finden.

Ergänzungen und Widersprüche

Risikobasiertes Testen und der **checklistenbasierte Ansatz** ergänzen sich gut: Die Risikoanalyse bestimmt, wo intensiver getestet werden muss, während Checklisten aus früheren Fehlerhistorien konkrete Testpunkte für genau diese Bereiche liefern.

Wiederverwendungsorientiertes Testen kann dem **risikobasierten Ansatz** widersprechen, wenn alte Testfälle nicht auf aktuelle Risikogebiete ausgerichtet sind und neue Risiken dadurch ungedeckt bleiben.

Mögliche Konflikte

- Ressourcenkonflikt: Risikobasiertes Testen fordert Testtiefe, Wiederverwendung priorisiert Schnelligkeit.
- Aktualitätsproblem: Checklisten und alte Testfälle spiegeln vergangene Fehler wider, nicht aktuelle Risiken.
- Priorisierungskonflikt: Checklisten regen gleichmäßige Abarbeitung an, risikobasiertes Testen fokussiert gezielt.

Bewertung für ein sicherheitskritisches System

Risikobasiertes Testen sollte klar priorisiert werden – bei sicherheitskritischen Systemen kann ein vorbeugender Ansatz sogar verpflichtend sein. Wiederverwendungsorientiertes Testen ist kritisch zu hinterfragen, da es neue Sicherheitsrisiken übersehen kann. Der checklistenbasierte Ansatz eignet sich nur als Ergänzung.

Aufgabe 2 - Zuordnung zu Teststufen

- Komponententest – Isolierter Test einzelner Klassen/Methoden, meist als Whitebox-Test
- Komponentenintegrationstest – Prüft Zusammenspiel und Schnittstellen zwischen Komponenten
- Systemtest – Testet das Gesamtsystem aus Kunden-/Anwenderperspektive inkl. nicht-funktionaler Anforderungen
- Abnahmetest – Formale Bestätigung durch den Kunden, dass Abnahmekriterien erfüllt sind

1. Überprüfung der Berechnung im Zahlungsmodul ->

Komponententests

Berechnungen sind interne Funktionen einer einzelnen Komponente. Der Komponententest prüft genau diese isolierte Funktionalität.

2. Kommunikation zwischen Warenkorb und Zahlungsmodul ->

Komponentenintegrationstest

Das Zusammenspiel zweier Komponenten und ihre Schnittstellen sind Gegenstand des Integrationstests. Auch wenn beide Komponenten einzeln fehlerfrei sind, können Schnittstellenfehler (Datenformate, Timing) auftreten.

3. Gesamter Bestellprozess aus Benutzersicht -> **Systemtest**

Der Bestellprozess nutzt alle Komponenten zusammen und ist erst auf Gesamtsystemebene beobachtbar. Der Systemtest prüft aus Kundenperspektive, ob alle Anforderungen erfüllt sind.

4. Validierung durch den Kunden -> **Abnahmetest**

Der Abnahmetest stellt das Urteil des Kunden in den Vordergrund. Der Kunde prüft, ob das System seinen Erwartungen und vertraglichen Kriterien entspricht.

Aufgabe 3 - Nur Systemtests durchführen

1. Auswirkungen

Fehlerfindung: Komponenteninterne Fehler und Schnittstellenfehler sind im Systemtest kaum isolierbar. Die Fehlerfindungsrate sinkt erheblich, da kein direkter Zugang zur Einzelkomponente besteht.

Testaufwand: Im Systemtest laufen alle Komponenten gleichzeitig. Wenn dort ein Fehler auftritt, weiß man nicht, wo er herkommt. Man muss also aufwändig rückwärts analysieren, welche Komponente das Problem verursacht hat

Debugging: Die Fehlerlokalisierung wird drastisch erschwert, da alle Komponenten gleichzeitig aktiv sind und unklar bleibt, welche die Fehlerursache trägt.

2. Mindestens 3 konkrete Probleme

- Nicht provozierbare Komponentenfehler: Ohne Zugang zur Komponentenschnittstelle können bestimmte Fehlerwirkungen nicht gezielt ausgelöst werden.
- Big-Bang-Effekt: Alle Fehler aller Schichten kumulieren sich; das System ist kaum lauffähig zu bringen.
- Späte Fehlerbehebung ist teuer: Fehler, die früh hätten gefunden werden können, werden erst spät entdeckt – die Korrekturkosten steigen deutlich.

3. Fazit

Das alleinige Durchführen von Systemtests spart kurzfristig scheinbar Ressourcen, führt aber zu höherem Aufwand, schlechterer Qualität und schwerer beherrschbaren Projektsituationen. Eine Kombination aller Teststufen ist effizienter.

Aufgabe 4 - Fehlerzuordnung zu Teststufen

Fehler in einer einzelnen Methode -> **Komponententest**

Eine einzelne Methode ist das klassische Testobjekt des Komponententests.

Fehler in der Schnittstelle zwischen zwei Modulen ->

Komponentenintegrationstest

Schnittstellenfehler entstehen in der Integration zweier Bausteine und können im Komponententest nicht entdeckt werden. Der Integrationstest deckt genau diese Fehlerklasse ab.

Schlechte Systemperformance -> **Systemtest**

Performanzprobleme resultieren aus dem Zusammenspiel aller Komponenten und sind erst auf Gesamtsystemebene messbar.

Missverständnis von Benutzeranforderungen -> **Abnahmetest**

Ob das System die tatsächlichen Nutzererwartungen erfüllt, zeigt sich erst im Abnahmetest.

Aufgabe 5 - Kritische Bewertung: Nur Integrations- und Systemtests

1. Kritische Bewertung

Der Vorschlag ist abzulehnen. Da man ohne Komponententests eine schwierige Fehlerfindung hat, dadurch zieht sich das Projekt in die Länge, kostet mehr Geld und die Qualität wird schlechter

2. Auswirkungen auf Qualität, Kosten, Risiko

- Qualität sinkt: Komponenteninterne Fehler bleiben unentdeckt
- Kosten steigen: Je später ein Fehler gefunden wird, desto teurer ist die Behebung. Frühes Testen spart Kosten nachweislich.
- Risiko steigt: Grundlegende Implementierungsfehler gelangen unentdeckt in den Systemtest und können zum Big-Bang-Effekt führen.

3. Empfehlung

Das vollständige Teststufenmodell (Komponenten- → Integrations- → System- → Abnahmetest) sollte beibehalten werden. Bei Ressourcenknappheit kann risikobasiert priorisiert werden: Komponententests für kritische Komponenten beibehalten, einfache Komponenten reduziert testen. Testtreiber aus dem Komponententest zahlen sich durch Wiederverwendung im Integrationstest aus.